THE FILE MANAGEMENT SYSTEM

BY

NAME: AJMAL
ROLL. NO: 45

NAME: Subhan Majeed
ROLL. NO: 18

NAME: M. Ehtisham Ashfaq
ROLL. NO: 25

Session 2023-2027

Version: 2
Submission Date: 20/02/2026

Department of Computer Science and Information Technology
Faculty of Computing and Engineering
University of Kotli Azad Jammu and Kashmir

THE FILE MANAGEMENT SYSTEM

By

NAME: AJMAL
Regd. No B230314045

NAME: Subhan Majeed
Regd. No B230314018

NAME: M. Ehtisham Ashfaq
Regd. No B230314025

A Report

Submitted with Fulfilment of the requirements of the Project Phase 2 in 5th Semester

Operating System

In

Information Technology

Session: 2023-2027

Department of Computer Science and Information Technology
Faculty of Computing and Engineering
University of Kotli Azad Jammu and Kashmir

Table of Content

FILE MANAGEMENT SYSTEM DESIGN PHASE 1
1. Introduction 2
2. Architectural Design Approach 2
2.1. Presentation Layer 3
2.2. Logic Layer 4
2.3. Application Execution Layer 5
2.4. Operating System Layer 6
2.5. Database and Storage Layer 7
1.3. Overall System Flow 8
1.4. Summary of the Design Phase 8

Table of Figures

Figure 1. File Management System Desing 1
Figure 2. Presentation Layer 3
Figure 3. Logic Layer..... 4
Figure 4. Application Execution Layer 5
Figure 5. Operating System Layer 6
Figure 6. Database and Storage Layer 7

FILE MANAGEMENT SYSTEM DESIGN PHASE

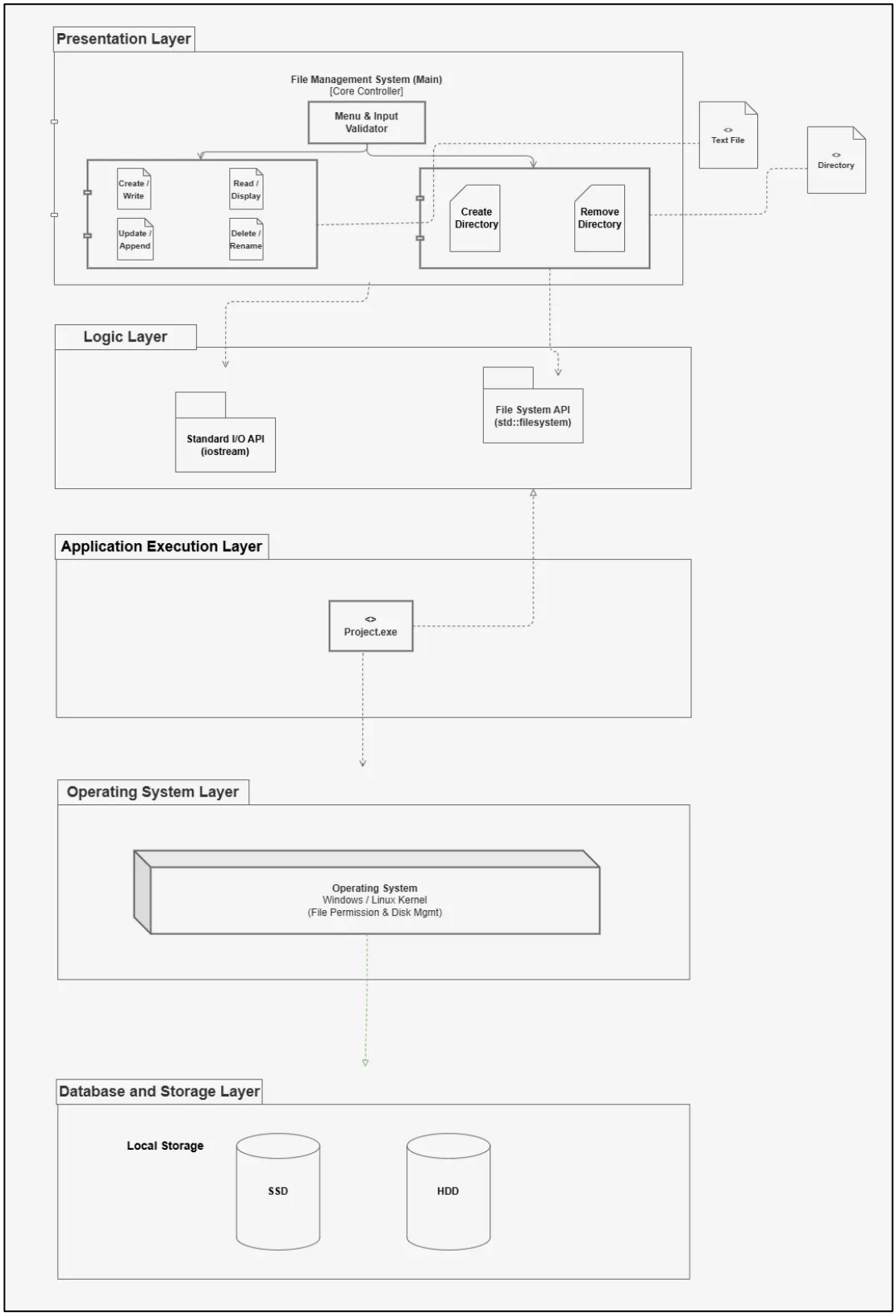

Figure 1. File Management System Desing

1. Introduction

The Design Phase is a critical stage in the Software Development Life Cycle (SDLC), where system requirements are transformed into a structured and well-defined architectural solution. In the context of the File Management System, this phase translates both functional requirements—such as file creation, reading, updating, deletion, and directory management—and non-functional requirements—such as performance, security, reliability, and maintainability—into a clear technical blueprint. Rather than focusing on what the system should do, the design phase defines how the system will accomplish these tasks. It establishes the overall system structure, defines component responsibilities, determines communication mechanisms between modules, and ensures that the system will operate efficiently within its intended environment.

For the File Management System, the design phase focuses on organizing the system into logical components that interact in a controlled and structured manner. This includes defining the main controller, input validation mechanisms, file and directory handling modules, API integration, operating system interaction, and storage management. By clearly identifying these components and their relationships, the system avoids tight coupling between modules and reduces overall complexity. The design ensures that each component performs a specific role, contributing to a cleaner implementation and easier debugging process. Additionally, the data flow between components is carefully structured to follow a top-down execution pattern, improving clarity and control over system operations.

2. Architectural Design Approach

The system is designed using a multi-layered architecture consisting of:

1. Presentation Layer
2. Logic Layer
3. Application Execution Layer
4. Operating System Layer
5. Database and Storage Layer

This architectural style ensures that each layer performs a specific responsibility while communicating with adjacent layers only.

2.1. Presentation Layer

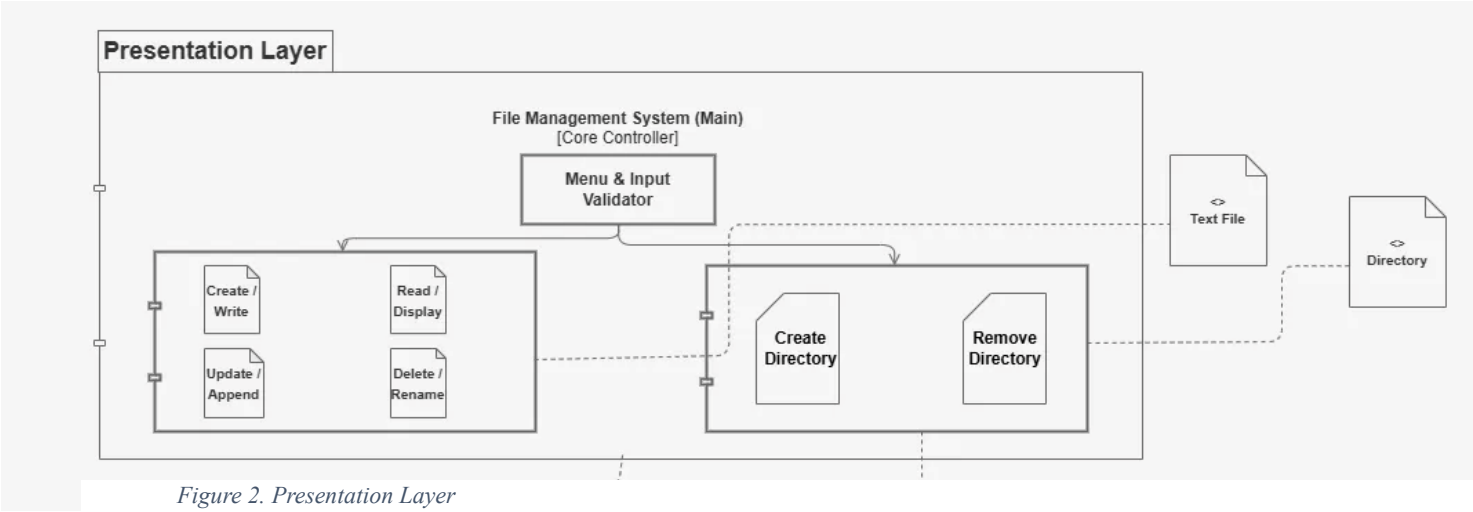

Figure 2. Presentation Layer

The Presentation Layer represents the user interaction interface of the File Management System. It is responsible for receiving user inputs, displaying outputs, and controlling the overall user experience.

At the center of this layer is the File Management System (Main) – Core Controller, which manages the execution flow of operations. The core controller connects to the Menu & Input Validator, ensuring that all user inputs are validated before being processed further. This prevents invalid commands and enhances system reliability.

This layer provides the following file-related operations:

- Create / Write File
- Read / Display File
- Update / Append File
- Delete / Rename File
- Create Directory
- Remove Directory.

Key Responsibilities:

- User input handling
- Input validation
- Menu navigation
- Command routing to logic layer

2.2. Logic Layer

Figure 3. Logic Layer

The Logic Layer serves as the core processing unit of the system. It translates validated user requests into actual file system operations using standard C++ libraries. It ensures that all operations adhere to system rules and prevents unauthorized access. Additionally, it manages error handling and returns appropriate status messages to the user interface. The layer also optimizes performance by efficiently handling multiple simultaneous requests.This layer includes:

- Standard I/O API (iostream)
  Handles reading and writing text data to and from files.
- File System API (std::filesystem)
  Manages directory creation, deletion, renaming, and file manipulation.

The Logic Layer acts as an abstraction layer between the Presentation Layer and the Operating System. It ensures modularity and separation of concerns, meaning that business logic is independent of user interface design.

Key Responsibilities:

- File processing logic
- Directory management
- Data manipulation
- Exception handling
- Communication with OS layer

2.3.Application Execution Layer

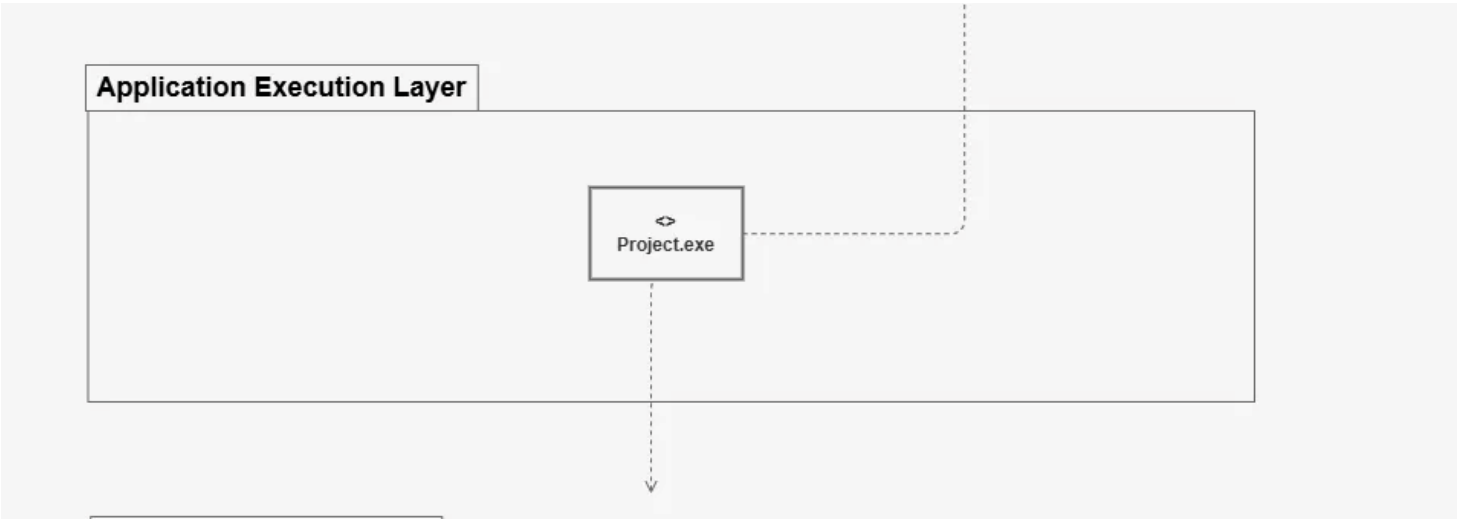

Figure 4. Application Execution Layer

The Application Execution Layer represents the compiled executable file (Project.exe). It serves as the interface between the user and the underlying system, initiating all operations when the application is launched. This layer loads necessary resources, initializes configurations, and starts the Logic Layer to process user commands. It also handles runtime events and communicates feedback to the user interface. Additionally, it ensures that the application runs smoothly by managing memory and system resources efficiently. Any errors encountered during execution are reported through this layer, maintaining system stability.This layer is responsible for:

- Running the application
- Managing runtime execution
- Initiating communication between layers
- Handling system-level process execution

It serves as the bridge between the program's internal logic and the operating system environment.

Key Responsibilities:

- Program execution
- Runtime management
- Process handling
- Triggering system calls

2.4.Operating System Layer

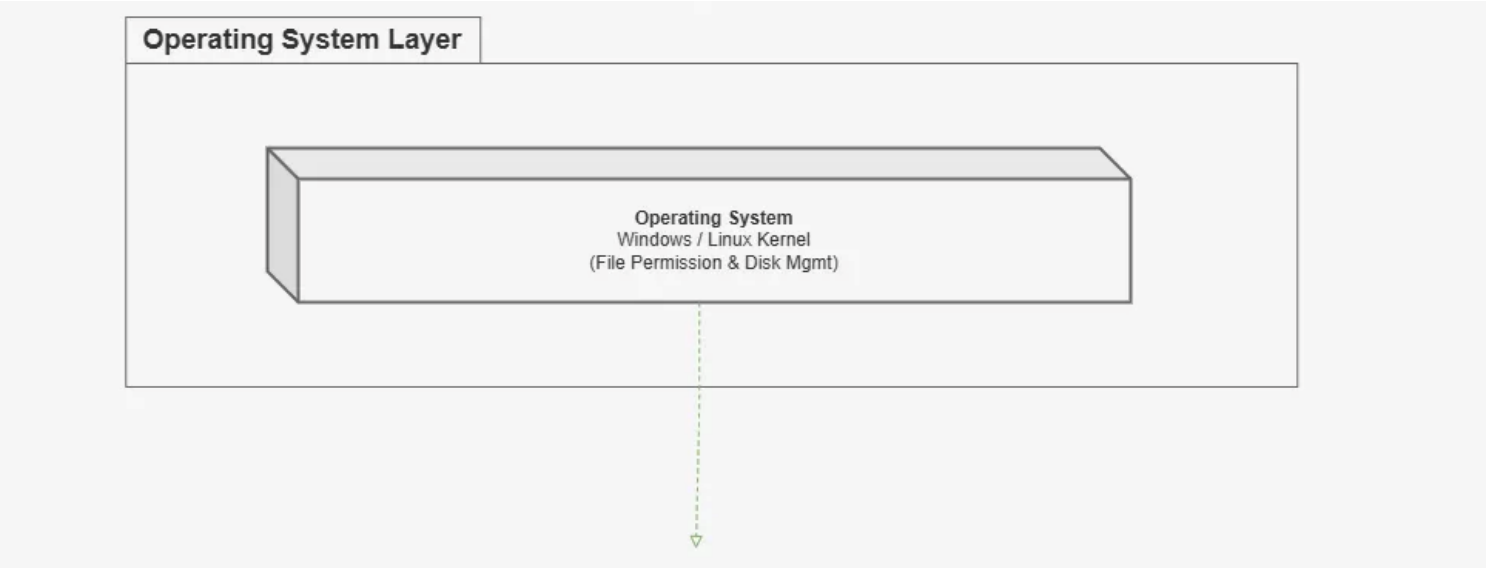

Figure 5. Operating System Layer

The Operating System Layer represents the underlying system kernel (Windows or Linux). It provides essential services such as memory management, process scheduling, and hardware abstraction to higher layers. This layer ensures secure and efficient communication between the hardware and application layers. It also manages input/output operations, file systems, and device drivers, enabling the application to perform tasks without directly accessing hardware. Additionally, it enforces system security policies and handles interrupts or exceptions that occur during execution. By coordinating resources and processes, the OS layer ensures overall system stability and performance.This layer provides low-level services required for file operations, including:

- File permission management
- Disk management
- Memory management
- Process control
- System call execution

The operating system ensures that file access follows security rules and that disk operations are performed correctly.

2.5.Database and Storage Layer

Figure 6. Database and Storage Layer

The Database and Storage Layer represents physical storage devices where files are permanently stored. It manages data persistence, ensuring that all information is reliably written to and retrieved from disks or other storage media. This layer handles data organization, indexing, and retrieval to optimize access speed and efficiency. It also enforces data integrity and consistency, preventing corruption or loss during read/write operations. Additionally, it supports backup and recovery mechanisms to safeguard critical data. By abstracting the complexity of physical storage, this layer allows higher layers to interact with data seamlessly without worrying about hardware specifics.In the diagram, this includes:

- **SSD (Solid State Drive)**
- **HDD (Hard Disk Drive)**

This layer is responsible for:

- Physical data storage
- Data persistence
- Long-term file retention
- Storage device management

The operating system communicates directly with this layer to perform read and write operations.

1.3. Overall System Flow

1. The user selects an operation from the Presentation Layer.
2. The input is validated by the Menu & Input Validator.
3. The request is passed to the Logic Layer.
4. The Logic Layer uses `iostream` or `std::filesystem`.
5. The request is executed through the Application Execution Layer.
6. The Operating System performs system-level operations.
7. Data is stored or retrieved from SSD/HDD storage.
8. The result is returned back to the user.

1.4. Summary of the Design Phase

The Design Phase of the File Management System converts requirements into a structured layered architecture. Each layer has clearly defined responsibilities, ensuring modularity, scalability, security, and maintainability. The system design supports efficient file handling while maintaining portability across operating systems.